

Python Language Programming

Introduction

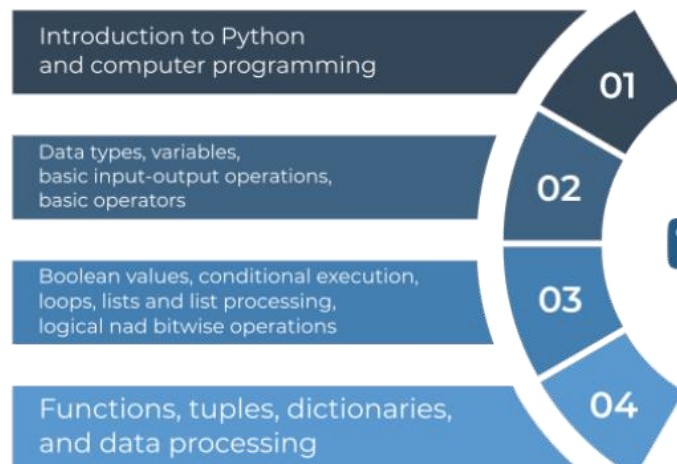
- **Disclaimer**

- These slides and notes are based on the contents of the courses:

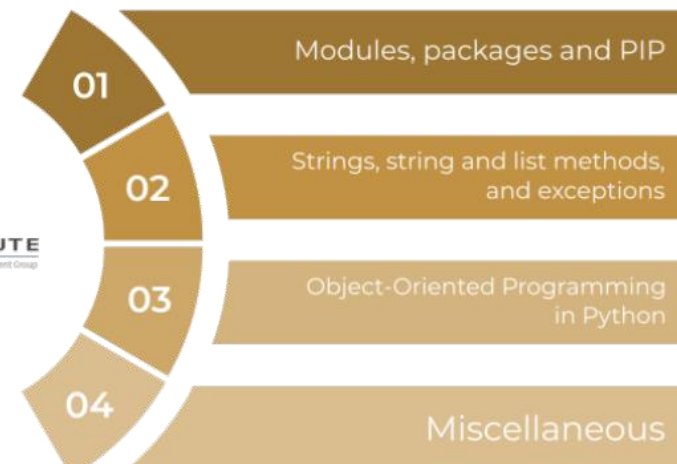
- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;
(www.pythoninstitute.org)
- offered in the Cisco Networking Academy program.

Python Essentials 1



Python Essentials 2



- **Disclaimer**

- These slides and notes are based on the contents of the courses:

- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;
(www.pythoninstitute.org)
- offered in the Cisco Networking Academy program.

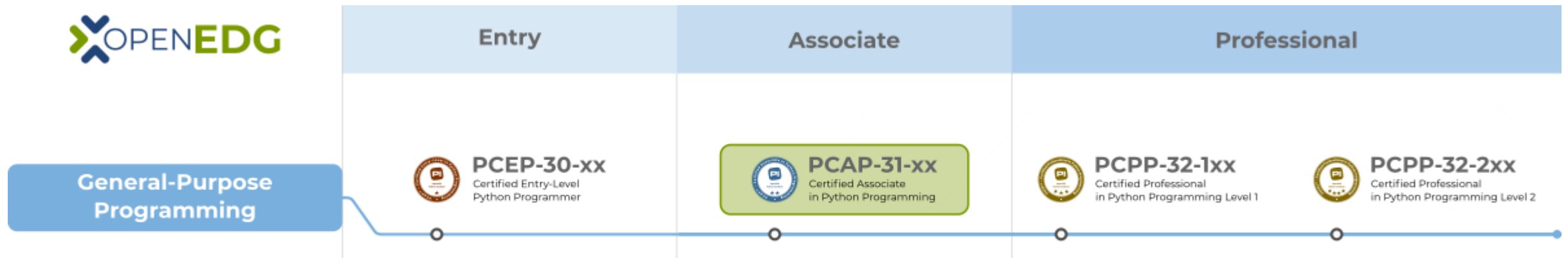


Table of Contents

- **Definition**
- **Advantages vs Disadvantages**
- **C Language vs Python Language**
- **Rank**
- **Compiled or interpreted?**
- **Versions**
- **Syntax**
- **Built-in Functions**
- **Modules**
- **Bibliography**

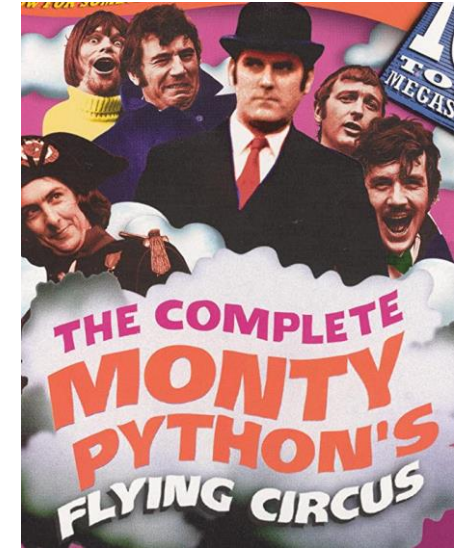
Python Language Programming

- **Definition**

- The name of the Python programming language comes from an old BBC television comedy sketch series called **Monty Python's Flying Circus**



- Python was created in 1991 by **Guido van Rossum**



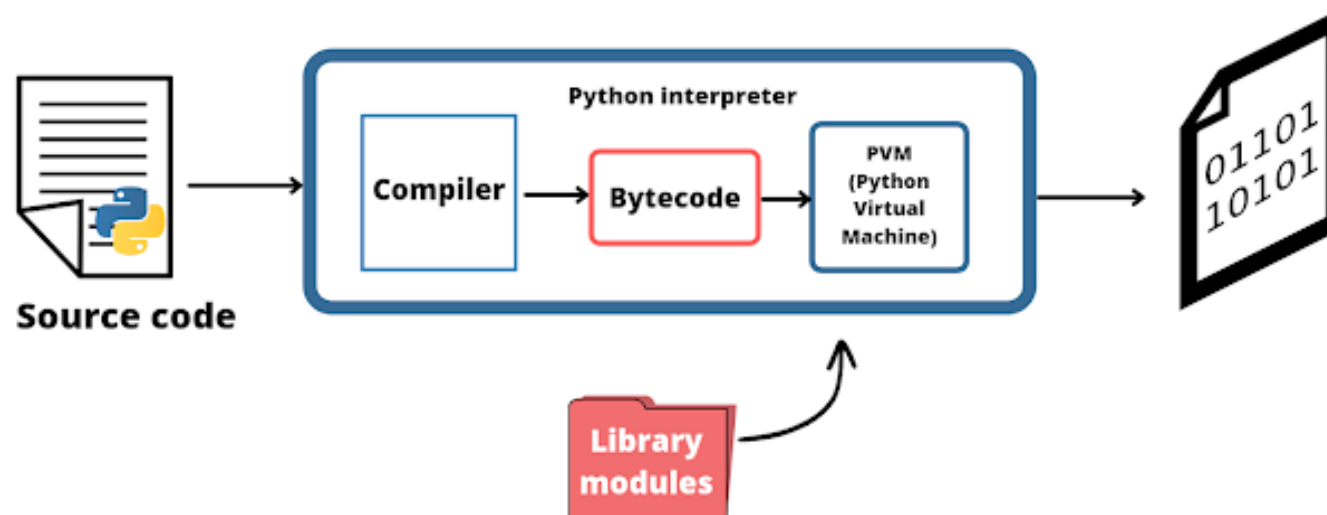
Guido
van
Rossum

Python Language Programming

- **Definition**

- Python is a programming language:
 - High level
 - Interpreted
 - (weak) Dynamic Typing
 - Object-oriented

Documentation:
<https://www.python.org/doc/>



Python Language Programming

- Advantages vs Disadvantages



- Drawbacks:

- does NOT deliver exceptional performance;
- may be resistant to some simpler testing techniques - debugging Python's code can be more difficult than with other languages;

Python Language Programming

- C Language vs Python Language

C

- Lower level
- Compiled
- Strong typing
- Non object-oriented

Explicit memory allocation
(pointers management)

Python

- Higher level
- Interpreted
- Weak typing
- Object-oriented

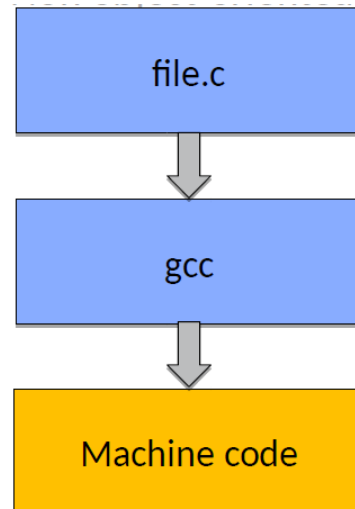
Implicit memory allocation
(hidden to the user)

Python Language Programming

• C Language vs Python Language

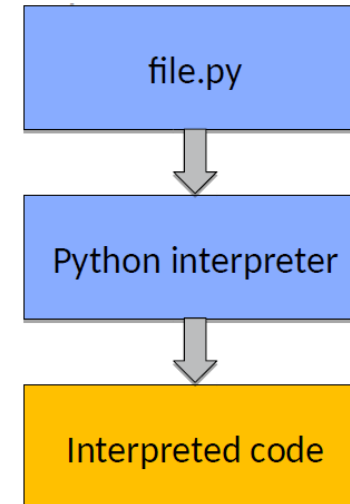
C

- Lower level
- Compiled
- Strong typing
- Non object-oriented



Python

- Higher level
- Interpreted
- Weak typing
- Object-oriented



https://www.youtube.com/watch?v=sv_6BXQqBc8

Python Language Programming

• C Language vs Python Language

	COMPILATION	INTERPRETATION
ADVANTAGES	• the execution of the translated code is usually faster; • only the user has to have the compiler - the end-user may use the code without it; • the translated code is stored using machine language - as it is very hard to understand it, your own inventions and programming tricks are likely to remain your secret.	• you can run the code as soon as you complete it - there are no additional phases of translation; • the code is stored using programming language, not the machine one - this means that it can be run on computers using different machine languages; you don't compile your code separately for each different architecture.
DISADVANTAGES	• the compilation itself may be a very time-consuming process - you may not be able to run your code immediately after any amendment; • you have to have as many compilers as hardware platforms you want your code to be run on.	• don't expect that interpretation will ramp your code to high speed - your code will share the computer's power with the interpreter, so it can't be really fast; • both you and the end user have to have the interpreter to run your code.

Python Language Programming

• C Language vs Python Language

C

- Lower level
- Compiled
- Strong typing
- Non object-oriented

```
int a;  
a = 5;    //Correct  
a = "string";  //Error!!
```

Type must be known by the compiler

Python

- Higher level
- Interpreted
- Weak typing
- Object-oriented

```
a = 5 #Correct  
a = "string" #Correct  
a = 1.3 #Correct
```

No explicit type declaration

Python Language Programming

• C Language vs Python Language

C

- Lower level
- Compiled
- Strong typing
- Non object-oriented

```
void sort(int * array){...}  
...  
int grades[] = {1, 6, 4, 8, 9};  
sort(grades);
```

Python

- Higher level
- Interpreted
- Weak typing
- Object-oriented

```
ada = [1, 6, 4, 8, 9]  
ada.sort()
```

sort is a method of the object `ada` and act on it

Python Language Programming

- Rank

Aug 2024	Aug 2023	Change	Programming Language		Ratings	Change
1	1			Python	18.04%	+4.71%
2	3	↑		C++	10.04%	-0.59%
3	2	↓		C	9.17%	-2.24%
4	4			Java	9.16%	-1.16%
5	5			C#	6.39%	-0.65%
6	6			JavaScript	3.91%	+0.62%
7	8	↑		SQL	2.21%	+0.68%

<https://www.tiobe.com/tiobe-index/>

Python Language Programming

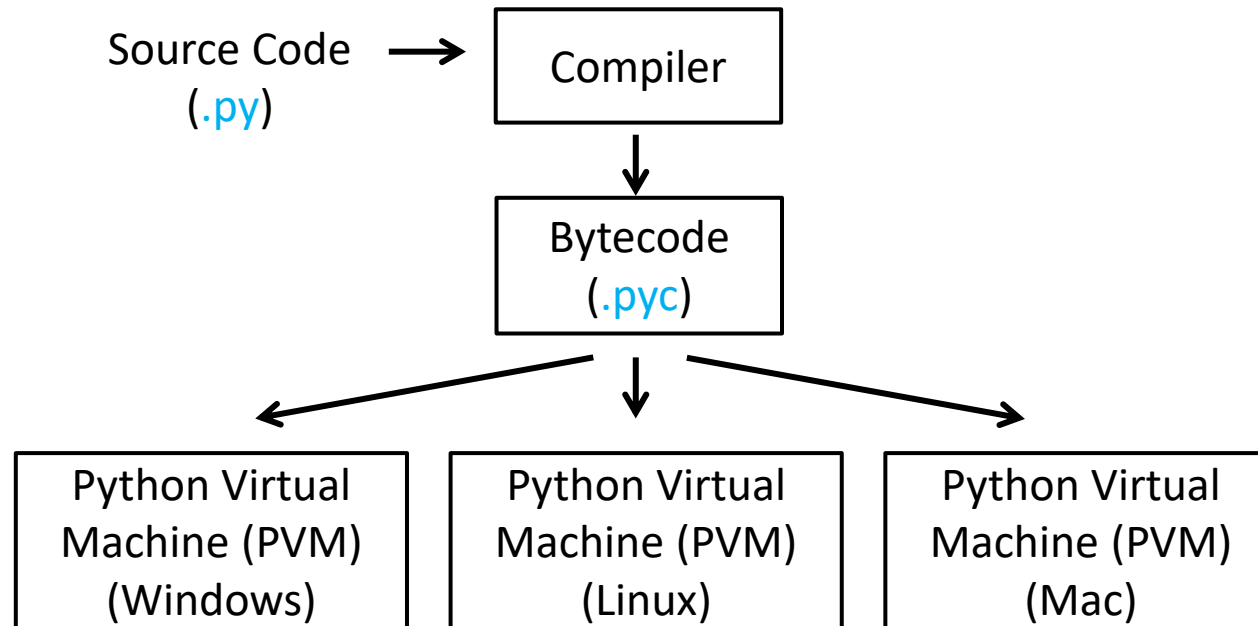
- Rank

Programming Language	2024	2019	2014	2009	2004	1999	1994	1989
Python	1	3	8	6	8	26	23	-
C	2	2	1	2	2	1	1	1
C++	3	4	4	3	3	2	2	2
Java	4	1	2	1	1	16	-	-
C#	5	6	5	7	7	21	-	-
JavaScript	6	7	9	9	9	18	-	-

<https://www.tiobe.com/tiobe-index/>

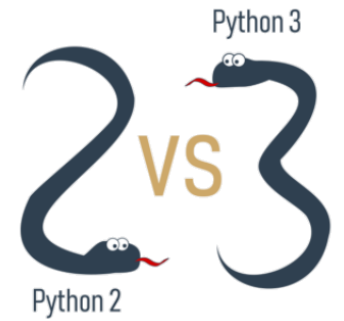
Python Language Programming

- **Compiled or interpreted?**
 - Python is both compiled and interpreted language



<https://www.youtube.com/watch?v=Korc0gwhUNE>

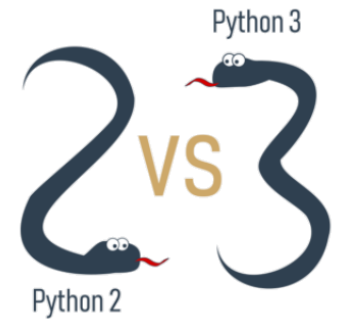
Python Language Programming



- **Versions**
- There are two popular versions of the Python programming language:
 - **Python 2 (older version)**
 - more conservative
 - resembles languages derived from the classic C language
 - **Python 3 (the newer)**
 - a completely different language, although very similar to its predecessor
 - newer versions of Python 3 are backwards compatible with the previous versions of Python 3

These two versions of Python aren't compatible with each other.

Python Language Programming



- **Versions**

- There are two popular versions of the Python programming language:

```
# Check the Python Version
```

```
import sys
```

```
print(sys.version)
```

```
luisamveloso@DESKTOP-3MV x + v
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

This message is shown once a day. To disable it please create the
/home/luisamveloso/.hushlogin file.
luisamveloso@DESKTOP-3MVA00D:~$ python3
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import sys
>>> print(sys.version)
3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0]
>>>
```

Python Language Programming

- **Versions**

- In addition to Python 2 and Python 3, there is more than one version of each:

- **Cpython:**

- default and most widely used implementation of the Python
 - written in C and Python
 - both an interpreter and a compiler as it compiles Python code into bytecode before interpreting it



- **Cython:**

- compiled language
 - allows to write Python code with C-inspired syntax extensions
 - produces C source files compatible with CPython



Python Language Programming

- **Versions**

- In addition to Python 2 and Python 3, there is more than one version of each:

- **Jython**

- designed to run on the Java platform
 - uses Java classes instead of Python modules
 - compiles Python source code to Java bytecode



Python Language Programming

- **Syntax**

- **alphabet**: a set of symbols used to build words of a certain language (e.g., the Latin alphabet for English)
- **lexis (aka a dictionary)**: a set of words the language offers its users
- **syntax**: set of rules (formal or informal) used to form a valid sentence (e.g., "I am a python" is a syntactically correct, while "I a python am" isn't)

```
>>> print("Hello World!")  
File "<stdin>", line 1  
    print("Hello World!")  
          ^
```

```
SyntaxError: EOL while scanning string literal
```

- **semantics**: set of rules determining if a certain phrase makes sense (e.g., "I ate a doughnut" makes sense, but "A doughnut ate me" doesn't)

Python Language Programming

- **Syntax**

- **Variables**

- do NOT need declaration before usage

```
var1=3.14  
var2=50  
var3="Hello World!"
```

- **Instruction lines**

- ‘;’ are NOT necessary to end an instruction line

- **Code blocks** (e.g if-else, for loops)

- use **‘:’ + indentation** instead of **‘{’**

- **INDENTATION IS FUNDAMENTAL!!!**

- Indentation is done with Tabs or Whitespaces

- **Comments**

- **#** (single line)
 - **"""** (multiline, docstring)

```
# Check the Python Version  
import sys  
print(sys.version) #this line prints a string
```

Python Language Programming

- **Built-in Functions**

- the Python 3.8 comes with 69 built-in functions:
 - always available and don't have to be imported
 - <https://docs.python.org/3/library/functions.html>
- `print()` function
 - It outputs a specified message to the screen/console window

```
>>> print("Hello World!")
Hello World!
>>> print("Hello \nWorld!")
Hello
World!
>>> print("Hello", "World!")
Hello World!
```

Python strings can be delimited with:

- double quotes, e.g., "I am a string"
- single quotes, e.g. 'I am a string, too'

Python Language Programming

- **Built-in Functions**

- `input()`
- able to read data entered by the user and to return the same data to the running program.

```
print("Tell me anything...")  
anything = input()  
print("Hmm...", anything, "... Really?")
```

=

```
anything = input("Tell me anything...")  
print("Hmm...", anything, "...Really?")
```

the prompt string: it allows to write a message before the user input

Python Language Programming

- **Built-in Functions**

- `input()`
- able to read data entered by the user and to return the same data to the running program.

```
name = input("Enter your name: ")  
print("Hello, " + name + ". Nice to meet you!")  
print("\nPress Enter to end the program.")  
input()  
print("THE END.")
```



```
Enter your name: Luis  
Hello, Luis. Nice to meet you!  
  
Press Enter to end the program.  
  
THE END.
```


Python Language Programming

- **Built-in Functions**

- `help()`
 - shows the object documentation (attributes, functions, behavior, ...)
- `len()`
 - returns the length of the object
 - NOT all objects have a length!
- `dir()`
 - list all attributes and functions of obj
- `type()`
 - returns the type of the object

```
>>> var1=3.14
>>> var2=50
>>> var3="Hello World!"
>>> type(var1)
<class 'float'>
>>> type(var2)
<class 'int'>
>>> type(var3)
<class 'str'>
```

Python Language Programming

- **Modules**

- As in C, it is possible to load libraries (modules) to extend functionalities
- Modules can be imported in various ways:

<code>import os</code> #import namespace of module os		<code>os.curdir</code> <code>os.listdir("Desktop")</code>
<code>import os as O</code> #import namespace of module os with an alias O		<code>O.curdir</code> <code>O.listdir("Desktop")</code>
<code>from os import *</code> #import the entire namespace		<code>curdir</code> <code>listdir("Desktop")</code>
<code>from os import curdir as CC</code> <code>from os import listdir as LS</code> #import a specific element of the module		<code>CC</code> <code>LS("Desktop")</code>

Bibliography

- Burkov, A. (2019). The hundred-page machine learning book. Andriy Burkov.
- **Cisco. (2022). Python Essentials. Retrieved September 4, 2024, from <https://www.netacad.com/>**
- Geron, A. (2019). Hands-on machine learning with scikit-learn, keras, and tensorflow. O'Reilly.
- Ekman, M. (2022). Learning Deep Learning. Addison-Wesley.
- Mckinney, W. (2017). Python for data analysis: Data wrangling with pandas, numpy, and ipython. O'Reilly.
- Raschka, S., & Mirjalili, V. (2019). Python machine learning: Machine learning and deep learning with python, scikit-learn, and tensorflow. Packt Publishing.
- Russel, S., & Norvig, P. (2018). Artificial intelligence: A modern approach. Pearson Education Limited.